

Test Planning: A Complete Guide for QA Success

“A goal without a plan is just a wish.”

In QA, a **test plan** is your roadmap to delivering a quality product confidently and efficiently.

What is a Test Plan?

A **Test Plan** is a detailed document that outlines the strategy, scope, objectives, schedule, resources, and activities required for testing a software product.

It defines:

-  What will be tested
 -  How to test it
 -  When it will be tested
 -  Who will test it
 -  What resources are needed
-

Pre-Requisites for Test Planning

Before creating a test plan, ensure you have:

1. **Requirements Doc (SRS/BRD)** – "No specs? Red flag!"
2. **Timeline** – Sync with dev milestones
3. **Tool Stack** – Pick the right weapons:
 -  Test Management: TestRail / Qase
 -  Automation: Selenium / Cypress / Playwright
 -  Bug Tracking: JIRA / Linear
4. **Team Kickoff** – Hold a quick sync with Devs + PM to align on features, scope, and priorities.

 **Why it matters:** These steps ensure your plan is based on facts, not assumptions.

Components of a Good Test Plan (with Calculator App Example)

1. Test Plan ID

 TP-CALC-001 (for Calculator App v1.0)

A unique identifier for traceability.

2. Objective

Clearly state what you're testing and why.

 Example:

- *To verify the accuracy, usability, and error handling of a basic calculator app.*

 **Why it matters:** Keeps the testing focused on business goals.

3. Scope

Defines what will and won't be tested

 Example:

-  In Scope: Addition, subtraction, multiplication, division, UI validation
-  Out of Scope: Scientific mode, theme settings, history logs

 **Why it matters:** Prevents scope creep and confusion during execution.

4. Test Strategy

 Describe how testing will be approached:

- Levels: Unit, Integration, System
- Types: Manual + Automation
- Techniques: BVA (Boundary Value Analysis), EP (Equivalence Partitioning), Smoke, Regression

 Example:

- “We’ll use a hybrid strategy: manual testing for UI and automation for basic math operations using Cypress. Focus on BVA and edge cases.”

5. Test Environment Setup

Detail where and on what the testing will happen. Include:

- URLs or app builds
- Browsers/devices
- Required tools (e.g. emulator, Postman)
- Access credentials

 Example:

 Environment: Staging environment with Android and iOS devices

- DB: None (fully client-side app)
- Tools: Android Emulator, iPhone 12 Simulator, Postman (for calculator API), Cypress (for web version)
- Access: GitHub build pipeline, TestFlight or Google Play Internal Testing

 **Why it matters:** A broken or incomplete environment = blocked testing.

6. Test Deliverables

 List what documents and outputs will be produced.

 Example:

- Test Plan Document
- Functional Test Cases (Excel/TestRail)
- Bug Reports (JIRA)
- Test Summary Report

 **Why it matters:** Helps everyone know what to expect at the end.

7. Entry and Exit Criteria

Define the conditions to **start** and **stop** testing.

 Entry:

- Build deployed to test devices
- Feature freeze completed
- Test data prepared

 Exit:

- All high/critical bugs fixed
- 100% test cases executed
- Summary report shared with stakeholders

 **Why it matters:** Avoids rushing in or shipping too early.

8. Schedule & Milestones

Activity	Start	End
Test Planning	Apr 1	Apr 2
Test Case Writing	Apr 3	Apr 5
Test Execution	Apr 6	Apr 12
Final Report	Apr 13	Apr 13

9. Roles & Responsibilities

Team Member	Role	Responsibility
Alex	QA Lead	Plan, review, stakeholder sync
XYZ	QA Engineer	Execute tests, log bugs
ABC	Automation QA	Build automation for core flows

10. Risks and Mitigation

Risk	Impact	Mitigation
Delayed environment setup	High	Align with DevOps early
Changing requirements	Medium	Weekly review with PM
Resource availability	Medium	Cross-train backup QA members

Real Example: Detailed Test Plan for Calculator App v1.0

1. Objective

To ensure users can perform basic arithmetic operations (+, -, ×, ÷) correctly, and that the calculator provides accurate outputs, handles invalid inputs gracefully, and maintains UI consistency across devices.

2. Scope

Category	In Scope	Out of Scope
Functionality	Addition, Subtraction, Multiplication, Division operations	History/saved calculations
UI	Button responsiveness, Layout validation	Themes/skins customization
Edge Cases	Division by zero handling, Overflow handling	Advanced math (sin, cos, log, etc.)

3. Test Types

- **Functional Testing:** Validate all core arithmetic operations.
- **UI Testing:** Verify button placement, label accuracy, and click responsiveness.
- **Error Handling Testing:** Test for invalid inputs like division by zero or large number overflows.

4. Entry & Exit Criteria

Phase	Criteria
Entry	Stable build deployed to QA environment- Test cases reviewed and approved by QA lead
Exit	100% of critical test cases passed- No open P1 (Critical) or P2 (Major) defects

5. Test Environment

Component	Details
Devices	iPhone 14, Pixel 7, iPad Air
OS / Browsers	iOS 16+, Android 13+, Chrome 120+, Safari latest
Tools	- TestRail (for Test Case Management)- JIRA (for Defect Tracking)- Cypress (for Web Automation)
Test Data	- Valid Inputs: (e.g., "5+3", "8×2")- Invalid Inputs: (e.g., "5/0", "999999999×999999999")

6. Deliverables

Before Testing:

- ☒ Test Plan Document (this document)
- ☒ 200+ Test Cases covering all operations
- ☒ Complete Valid/Invalid Input Data Sets

During Testing:

- ☒ Daily Execution Status Logs
- ☒ Defect Reports (JIRA) with screenshots and/or videos
- ☒ Test Execution Summary Updates

After Testing:

- ☒ Final Test Summary Report (pass/fail rates, defect metrics)
 - ☒ Release Readiness Memo/Recommendation
-

7. Risks & Mitigation

Risk	Mitigation Strategy
Device-specific UI bugs	Test on at least 3 real devices (mix of iOS and Android)
Flaky automation tests	Implement retry logic in Cypress test scripts
Tight project deadline	Prioritize critical path tests (basic operations first) and de-scope lower priority cases if needed

Quick Summary for the Calculator App v1.0

Section	Example
Objective	Validate arithmetic operations
Scope	Basic operations, UI; exclude scientific mode
Test Strategy	Functional, UI, and Error handling tests
Environment	Android/iOS/Web, TestRail, Cypress
Deliverables	Test Cases, Bug Reports, Summary Report
Entry Criteria	Build ready, requirements locked
Exit Criteria	No critical defects, 100% critical test pass
Key Risks	Build delays, UI misalignment



Tips from a Senior QA

-  **Start Early** – Begin planning even before development finishes
-  **Involve Dev/PO** – Make it collaborative
-  **Keep it Visual** – Tables, timelines, charts help
-  **Review Often** – Keep your plan flexible and current

Final Thoughts

A **strong test plan** is more than a document — it's a **strategy for success**. It aligns teams, sets expectations, and ensures quality.

With this structured approach, you're not just testing software — you're building confidence.

The End